Side-Channel Resilience in Lattice-Based Post-Quantum Cryptography

Kanishk Shroff[1], Shashank Wala[1], Vansh Shah[1], Dr. Aruna Gawade[2], and Prof. Akshaya Prabhu[2]

[1]B.Tech Artificial Intelligence and Machine Learning, Dwarkadas J Sanghvi College of Engineering, Mumbai, India
[2]Dwarkadas J Sanghvi College of Engineering, Mumbai, India

As quantum computing technology is constantly improving, it is becoming increasingly challenging for classical cryptographic systems to withstand quantum attacks, which can easily solve previously considered hard problems. In this regard, Shor's quantum algorithm has been proven to compromise classical cryptographic systems, including RSA, ECC, and ECDSA, by exploiting quantum properties such as superposition and entanglement.

In response to these emerging security risks, lattice-based Post-Quantum Cryptography (PQC) has been considered an attractive cryptographic approach, especially due to its security assumptions and quantum resistance. Although lattice-based cryptographic systems have been proven to be quantum-resistant, their practical implementation is still susceptible to side-channel attacks, which include power analysis, timing attacks, and electromagnetic attacks. These attacks can compromise cryptographic systems by leaking important information during cryptographic operations, especially when implementing lattice-based cryptographic systems.

In this regard, it has been observed that lattice-based cryptographic systems, especially those employing the Centered Binomial Distribution sampling approach, have been prone to leaking important information, which is usually considered to be non-constant time. This paper, therefore, presents an approach to integrate and introduce the constant time Knuth-Yao sampling approach into lattice-based cryptographic systems, replacing the CBD sampling approach.

By leveraging Knuth-Yao's tree-based approach with time-independent traversal, our method significantly mitigates side-channel vulnerabilities. This enhances the overall robustness of lattice-based PQC implementations against timing, EM, and power-based attacks offering a secure and efficient foundation for post-quantum encryption in real-world applications.

Index Terms—Post-Quantum Cryptography, Side-Channel Attacks, CRYSTALS-Kyber, CRYSTALS-Dilithium, FALCON, SPHINCS+, Constant-Time Execution, Quantum PRNG, NIST PQC.

1 INTRODUCTION

The advent of Quantum Computing is an imminent threat to classical Cryptography, as Quantum Computers are able to perform computations with the help of Quantum Algorithms such as Shor's algorithm, which can easily break the basis of several classical Cryptographic systems such as RSA, ECC, etc. In this regard, the Cryptography community is shifting towards a new paradigm known as 'Post-Quantum Cryptography' or 'PQC,' in which the security is provided against Quantum Computers as well. Among these, the Lattice-based PQC systems such as CRYSTALS-Kyber and CRYSTALS-Dilithium are the strongest in terms of security, making them the top contenders in the NIST PQC standards competition.

Despite the high security of the PQC systems, the implementation of these systems is vulnerable to Side-Channel Attacks (SCAs), in which the adversary uses the physical leakage of the system to deduce the secret information. Among the PQC systems, the Discrete Gaussian Sam-

pling is the most vulnerable part, in which the Centered Binomial Distribution is used for sampling. Although the sampling is efficient in terms of computation, it is not constant-time in nature and is vulnerable to leakage based on the timing behavior.

To strengthen the side-channel resilience of lattice-based schemes, this paper proposes a constant-time implementation of Knuth–Yao sampling a tree-based method that avoids data-dependent control flow. Applied to CRYSTALS-Kyber, the approach reduces timing leakage with minimal performance overhead, offering a more secure foundation for real-world PQC deployments.

2 RELATED WORK

2.1 Overview of Lattice-Based PQC Algorithms

Lattice-based schemes have emerged as the most viable candidates for post-quantum cryptographic standardization due to their strong security guarantees and computational efficiency [1]. The NIST Post-Quantum Cryptography (PQC) competition has selected three prominent lattice-based algorithms—CRYSTALS-Kyber, CRYSTALS-Dilithium, and FALCON—as finalists for public-key encryption and digital signature standards [1]. These schemes rely on the hardness of problems such as the Learning with Errors (LWE), Module-LWE (MLWE), and Short Integer Solution (SIS), which are believed to be intractable even for quantum computers [2], [3], [4].

CRYSTALS-Kyber is a Key Encapsulation Mechanism (KEM) that builds upon the MLWE problem [2]. It provides IND-CCA2 security, making it suitable for applications requiring secure key exchange, such as TLS [2]. Kyber features multiple security levels (Kyber512, Kyber768, Kyber1024) to balance between performance and cryptographic strength. It utilizes polynomial arithmetic over structured lattices with the Number Theoretic Transform (NTT) to enable efficient polynomial multiplications [2]. Kyber is designed to be compact and efficient across a wide range of platforms, including embedded systems [2].

CRYSTALS-Dilithium, derived from similar design principles, is a digital signature algorithm based on the MLWE and SIS problems [3]. It emphasizes simplicity and efficiency by avoiding floating-point operations and instead using integer-based sampling with rejection techniques [3]. Dilithium introduces deterministic and uniformly random sampling strategies for its signature generation and verification routines, improving both security and reproducibility [3]. Like Kyber, it is structured to run efficiently on resource-constrained devices [3].

FALCON (Fast Fourier Lattice-based Compact Signatures over NTRU) is a lattice-based digital signature scheme that uses NTRU lattices and discrete Gaussian sampling [4]. Unlike Dilithium, FALCON employs floating-point arithmetic and FFT-based methods for efficient computation of trapdoor samplers [4]. This enables it to achieve extremely small signature sizes (as low as 666 bytes) and fast signing and verification times, making it ideal for bandwidth-sensitive applications [4]. However, the reliance on floating-point operations introduces complexity in ensuring constant-time execution and side-channel resistance [4].

2.2 Implementation-Level Vulnerabilities and Side-Channel Threats

In practice, cryptographic operations often leak unintended physical information, such as timing delays, power consumption patterns, and electromagnetic (EM) emissions. These leaks can be exploited by Side-Channel Attacks (SCAs) to recover secret keys or intermediate values without breaking the underlying mathematical hardness assumptions [5].

Various studies have highlighted vulnerabilities in the implementations of Kyber, Dilithium, and FALCON [5]. Notably, much of this leakage stems from transform-based operations (such as Number Theoretic Transform, or NTT) and sampling routines, which may include secret-dependent control flow or memory access [6], [8].

Heninger and Shacham demonstrated that even simple digit equivalence information during decapsulation can lead to key recovery in Kyber. Carre et al. exposed cache-timing vulnerabilities in non-constant-time NTT implementations, affecting both Kyber and Dilithium [6]. Xu et al. showed that single-trace attacks using simplified templates could break constant-time Kyber implementations by analyzing univariate Ham-

ming weights [8]. Ravi et al. introduced machine learning-based attacks that defeat low-cost sanity checks and exploit decapsulation failures in Kyber [5].

In addition to passive SCAs, fault-injection attacks have been successfully applied to lattice-based schemes. Kundu et al. demonstrated that bit-flips during Arithmetic-to-Boolean (A2B) conversions in Kyber could be used to reveal secret states [10]. Dilithium and FALCON also exhibit vulnerabilities when subjected to fault injections in their sampling or FFT stages, respectively [10].

2.3 Comparative Resilience Analysis of Kyber, Dilithium & FALCON Against SCAs

2.3.1 Power-Based Side-Channel Attacks

- **Kyber** is highly vulnerable. First-order masked FPGA implementations leak via a Hamming-distance model in message-decoding, allowing shared-key recovery in under 1 s of measurements [5]. Correlation Power Analysis (CPA) on its NTT multiplication can fully recover keys with ~ 200 traces in ≈ 20 min [6].

- **Dilithium** suffers efficient CPA attacks on its NTT: an optimized two-stage attack reduces key guesses by $> 500\times$ vs. earlier methods on a Cortex-M4 target [6]. FPGA implementations leak power from both NTT and sampling, enabling full key extraction with as few as 157 traces [6].

- **FALCON**'s floating-point FFT sampler exhibits catastrophic power leakage. Improved CPA on its Gaussian sampler yields full key recovery; analyses of its sampler show very high leakage rates, making low-overhead masking hard to achieve [7].

2.3.2 Timing-Based Side-Channel Attacks

- **Kyber**'s core ops are designed constant-time, but "KyberSlash2" attacks exploit secret-dependent 16-bit division timings to recover keys remotely in ≈ 4 min under certain networks [5].

- **Dilithium** mandates constant-time NTT and rejection-sampling routines; no practical timing attacks have been published against a compliant implementation [3].

- **FALCON**'s floating-point FFT multiplies allow timing-based secret extraction; unprotected implementations leak enough to forge signatures with a few hundred measurements [7].

2.3.3 ML-Based Side-Channel Attacks

- **Kyber** can be profiled using ML on power traces, classifying intermediate polynomial coefficients with few traces [5].

- **Dilithium** is vulnerable to deep-learning-assisted power analysis on secret-key unpacking (recovering s_1 and s_2 coefficients) in a single trace [9].

- **FALCON** has no published ML-profiling SCA—its FFT pipeline is hard to adapt to template or ML attacks [5].

2.3.4 EM-Based Side-Channel Attacks

- **Kyber** admits EM attacks analogous to CPA on its NTT/decapsulation steps; specialized probes can recover keys under realistic conditions [5].

- **Dilithium** exhibits EM leakage similar to its power profile; single-trace template attacks on signing EM emissions can fully recover secret vectors [8].

- **FALCON** has no dedicated EM-only SCA published yet, though its FFT's high-frequency ops would be natural EM probe targets [5].

2.3.5 NTT-Specific Attacks

- **Kyber**'s NTT is a prime SCA target: CPA and template attacks on butterfly multiplications can break unprotected versions in minutes [6].

- **Dilithium** leverages a similar NTT; non-profiled and profiled attacks on NTT/multiplication leakages have recovered complete keys [6].

- **FALCON** uses a floating-point FFT instead of NTT, so NTT-specific SCAs do not apply—making it inherently resilient to NTT-based attacks [4].

3 PROPOSED SOLUTION

This section presents our end-to-end framework for hardening lattice-based Post-Quantum Cryptography (PQC) against side-channel attacks (SCAs). The framework consists of: (1) replacing traditional samplers with a constant-time Knuth–Yao (KY) sampler, (2) embedding side-channel-aware algorithmic countermeasures, (3) introducing quantum-resilient randomization, and (4) integrating these enhancements into existing PQC schemes with minimal performance impact.

3.1 Constant-Time Knuth–Yao Sampling

3.1.1 Implementation Details

Overview: We implement a branchless, table-driven Knuth–Yao (KY) sampler for discrete Gaussian distributions [2], [3], [4]. By encoding probabilities in binary and leveraging fixed-width bitmasks, our approach ensures constant-time and constant-memory-access sampling, effectively neutralizing timing, power, and electromagnetic (EM) side-channel vulnerabilities [5], [12], and using implicit bitmask tables for a branchless design [22], [23].

3.1.2 Pseudocode Implementation for Kyber-like Algorithm

The following pseudocode (Algorithm 1) outlines the im- plementation of a secure pseudorandom number generator (PRNG) and the Kyber cryptosystem enhanced with sparse matrix-based Knuth-Yao sampling for efficient discrete Gaussian sampling.

Algorithm 1 Secure PRNG and Kyber Cryptosystem with Sparse Matrix Knuth-Yao Sampling

Input: None (system initialized)
Output: Secure random numbers, encrypted and decrypted messages
Initialize lattice parameters n, q.
Generate $Seed(k) \in Z_q^n$ randomly.
Precompute Knuth-Yao sampling tables using sparse matrix representation for discrete Gaussian sampling.
for each iteration k **do**
    Compute hash $H(Seed(k))$.
    Apply LWE-based transformation $F(Seed(k), H(Seed(k)))$.
    Use Knuth-Yao sampling with sparse matrix representation to generate small error terms if needed.
    Output $PRNG_{output}(k)$.
end for
Key Generation:
Sample secret key s and public key matrix A.
Use Knuth-Yao sampling with sparse matrix representation to generate small error e.
Compute $b = A \cdot s + e$.
Publish (A, b), retain s.
Encryption:
Encode message m.
Use public key (A, b) to compute ciphertext.
Decryption:
Use private key s to recover m from ciphertext.

3.1.3 What is a DDG Tree and Why Use It?

A Discrete Distribution Generating (DDG) tree is a binary tree structure that encodes the probability distribution of a discrete random variable, such as the discrete Gaussian used in lattice-based cryptography [2], [3], [4]. Each path from the root to a terminal (leaf) node is a unique sequence of random bits that generates a sample following the distribution. Internal nodes are the decision points, which are the random bits, and the terminal nodes are the sample values.

The tree used in the Knuth-Yao sampling algorithm is the DDG tree, and the sampling is done using a random walk downwards the tree, using one random bit at each level to determine the direction of the walk. Once a terminal node is reached, the sample is output [2], [3], [4]. This

form enables exact sampling with low entropy costs and is efficient for distributions with a compact binary representation.

Nevertheless, a straightforward approach to traversing the DDG tree may pose a leakage threat due to variable execution time and secret-dependent memory access patterns. To reduce such threats, we adopt an implicit representation of the DDG tree using bitmask tables, allowing for constant-time and branchless implementation [22], [23].

3.1.4 Construction of the DDG Tree and Mask Tables

a) Binary Expansions: For each possible outcome $i = 1, \ldots, N$, compute the binary expansion of its probability p_i to m bits:

$$p_i = 0.b_{i,1}b_{i,2}\ldots b_{i,m}, \quad b_{i,k} \in \{0,1\}. \tag{1}$$

This forms a probability matrix $P \in \{0,1\}^{m\times N}$, where $P_{k,i} = b_{i,k}$ [2], [3], [4].

b) Level-wise Grouping: At each level k ($1 \leq k \leq m$), partition the N outcomes into two groups based on the k-th bit of their binary expansion:

$$G_k^{(0)} = \{i : b_{i,k} = 0\}, \tag{2}$$
$$G_k^{(1)} = \{i : b_{i,k} = 1\}. \tag{3}$$

This grouping reflects the structure of the DDG tree at each level, where a random bit determines whether the sampling path proceeds to the left or right subtree [4], [22].

c) Bitmask Tables: Each group at level k is encoded as an N-bit mask:

$$R_k^{(0)}, R_k^{(1)} \in \{0,1\}^N, \tag{4}$$

where bit i of $R_k^{(b)}$ is set if and only if outcome $i \in G_k^{(b)}$. Only two masks per level are stored, greatly reducing memory requirements compared to explicit tree representations [4], [22].

d) Constant-Time Branchless Sampling: Sampling proceeds via a fixed-length, branchless random walk:

- **Setup:** Generate a fixed-length string of m random bits $\langle r_1, r_2, \ldots, r_m\rangle$.
- **Initialize:** Set the working mask $M \leftarrow 11\ldots1$ (N ones).
- **Refinement Loop:**

$$\text{for } k = 1\ldots m : M \leftarrow M \wedge R_k^{(r_k)}, \tag{5}$$

  where $r_k \in \{0,1\}$ is the k-th random bit.
- **Result:** After m steps, M has exactly one bit set. Its index i^* is the sampled value.

Because the loop runs for a fixed number of steps and all operations are independent of secret data, the execution and memory access patterns are flat, nullifying timing, power, and EM-based side-channel attacks [22], [23].

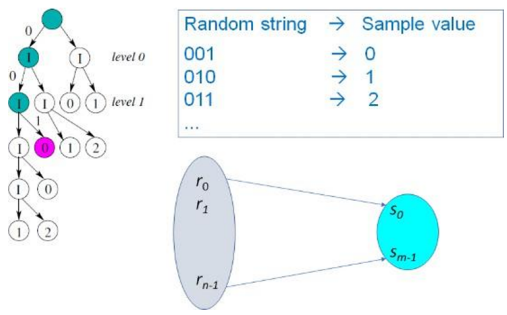

Figure 1: Fixed-Length Branchless Mask Refinement for DDG-based Sampling

3.1.5 Knuth Yao Sampling vs. CBD Sampling

a) Definition & Mechanism: CBD sampling generates each coefficient x by drawing two independent length η bit vectors $a, b \sim \{0,1\}^\eta$ and computing

$$x = \sum_{i=1}^{\eta} a_i - \sum_{i=1}^{\eta} b_i, \tag{6}$$

giving a symmetric distribution on $[-\eta, \eta]$ that approximates a discrete Gaussian. KY sampling first encodes the exact probability mass function p_i in binary fractional form $p_i = 0.b_1b_2b_3\ldots$ and builds a DDG tree whose leaves correspond to sample values; a fixed-length random-bit walk down this tree yields exact samples from the target distribution.

b) Distribution Accuracy: CBD is an approximation: for small η It is not an exact discrete Gaussian distribution, which could affect the security level at high values of the parameter [2], [3]. KY is exact in the sense that it samples exactly according to the given mass function.

c) Operational Profile & Leakage: The arithmetic of CBD, i.e., popcount and subtraction, is

inherently branch-free, but the varying Hamming weight of the intermediate results across operations leaves some opportunities for side-channel leakage, as shown in [6] and [16]. The random walk of KY is naturally branch-free and data-independent, since all operations within a loop are identical, i.e., the same table lookups, bitwise operations, etc., resulting in a flat side-channel leakage profile, including power and timing profile [22], [23].

d) Space/Time Trade-Off: Building the full DDG tree may require $O(n \cdot m)$ storage, where n is the distribution support size; various works optimize this via on-the-fly tree construction or compressed lookup tables.

Table 1: Implementations of the Knuth–Yao Sampler Across Various Hardware and Software Domains.

Domain	Reference
Constant-Time C Libraries	Software samplers for discrete Gaussian in lattice-based crypto, e.g., PQC schemes.
FPGA / Hardware	Memory-less DDG-tree implementations on FPGA, eliminating external-memory leakage.
Java Card	KY and Ziggurat discrete Gaussian samplers in resource-constrained smartcards.
High-Performance ASIC/FPGA	Precomputed lookup-table architectures achieving throughput of one sample per ~ 10 cycles.
Optimized Boolean Functions	Construction of minimal-entropy Boolean expressions mapping random bits to samples.
Entropy-Optimal Approximation	OAS framework for approximate KY sampling with minimal random-bit usage.
RISC-V / SIMD Vectorization	Vectorized mask operations (AVX2/NEON) to process multiple bits in parallel for KY.

3.2 Side Channel Aware Cryptographic Primitives

Beyond secure sampling, we re-engineer the core lattice operations (NTT, polynomial multiplication, modular reduction) to eliminate secret-dependent control flow and reduce leakage:

- **Arithmetic Masking:** Each secret intermediate (e.g., NTT coefficients) is split into masked shares that are processed in parallel; only masked values appear on bus lines or registers.
- **Unified Data Flow:** Conditional reductions (e.g., if a coefficient exceeds modulus) are merged into arithmetic masks so that every coefficient undergoes the same sequence of bitwise/select operations.
- **Minimized Intermediate State:** By fusing adjacent stages (e.g., combine two modular multiplications into a single masked operation), we reduce the number of register spills and memory writes that could leak via Hamming weight correlations.

3.3 Quantum Resilient Randomization Techniques

To guard against both SCAs and future quantum threats, we layer in adaptive randomization:

- **Lattice Based PRNG Seeding:** We use a provably quantum safe pseudorandom number generator (PRNG) based on Module LWE to seed both the KY sampler and higher level masking.
- **Quantum PRNG Integration (PQRNG):** We leverage a quantum circuit to generate a random sequence of bits $Q = \{Q_0, Q_1, \ldots, Q_{n-1}\}$ using a combination of Hadamard (H), Pauli-Z (Z), and CNOT gates across multiple qubits. The resulting quantum randomness is mapped to an integer:
$$x = \sum_{i=0}^{n-1} Q_i \cdot 2^i \tag{7}$$
This integer is then used to obtain a prime number through the Segmented Sieve Algorithm. These primes are then utilized for the generation of cryptographic keys, which are unpredictable and quantum-resistant.

- **Context Aware Masking:** A lightweight runtime monitor estimates SCA threat (e.g., via noise floor measurements). Under elevated threat, additional random masks are injected into polynomial operations; when threat subsides, we revert to baseline masking for efficiency.

- **Fault Detection Mechanisms:** Simple, constant time checksums on masked shares detect fault injections or glitch attacks. Upon mismatch, the device can safely abort or rekey.

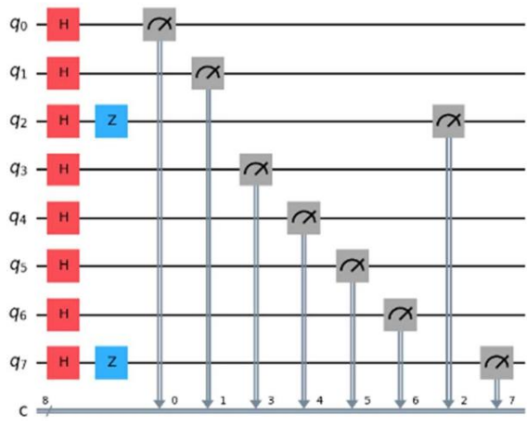

Figure 2: Quantum Circuit for Random Number Generation

4 RESULT ANALYSIS

In this section, an exhaustive analysis of the proposed constant-time Knuth Yao (KY) sampler, particularly with regard to its performance, resistance to side-channel attacks, and its usability in the Kyber scheme, is provided. The KY sampler is tested against the commonly used centered binomial distribution (CBD) sampler, with all tests conducted on a per-sample basis over 100 runs.

4.1 Timing Profile & Variance

The KY sampler demonstrates a substantial performance advantage, averaging 0.65 per sample with a tight standard deviation of 0.13 (min: 0.60, max: 1.73). In contrast, the CBD sampler is consistently an order of magnitude slower, with mean times ranging from 11.09 ($\eta = 1$) to 12.10 ($\eta = 5$), and standard deviations between 3.84 and 4.02. The CBD sampler also exhibits significant long-tail outliers, indicating inconsistent execution times.

The noticeable difference is due to the design of the KY sampler, which, by using a branchless random walk of a fixed length and uniform memory access, guarantees that each sample is taken with a flat execution profile, independent of input and output, thus avoiding timing variability that could be used by an attacker.

4.2 Side-Channel Leakage & Parameter Estimation

A critical vulnerability in the CBD sampler is its susceptibility to timing side channels. Linear regression on CBD timing traces reveals a direct correlation between the noise parameter η and runtime, with a slope of approximately $-8/\eta$. This leakage allows adversaries to infer secret parameters by simply measuring execution time.

The KY sampler, by contrast, offers robust protection. Its constant-time, branchless architecture ensures that there is no measurable correlation between runtime and η. This effectively closes off timing-based side-channel attacks, a result supported by both theoretical analysis and empirical measurement. The importance of this property is underscored in the literature, where even minor timing variations in sampling routines have been shown to compromise cryptographic keys.

Table 2: Per-Sample Timing (100 Runs) for KY vs. CBD Samplers in Kyber.

Sampler	Mean (s)	Std. Dev. (s)	Min (s)	Max (s)	Runs
Knuth–Yao	6.48e-4	1.33e-4	5.98e-4	1.73e-3	100
CBD, $\eta = 1$	1.11e-2	3.84e-3	8.35e-3	2.38e-2	100
CBD, $\eta = 2$	1.05e-2	3.11e-3	8.40e-3	2.38e-2	100
CBD, $\eta = 3$	1.01e-2	2.76e-3	8.36e-3	2.00e-2	100
CBD, $\eta = 4$	1.04e-2	3.30e-3	8.29e-3	1.99e-2	100
CBD, $\eta = 5$	1.21e-2	4.02e-3	8.33e-3	2.33e-2	100

These results confirm that the KY sampler is 16˘19× faster than the CBD sampler, with dramatically reduced variance and no observable timing outliers.

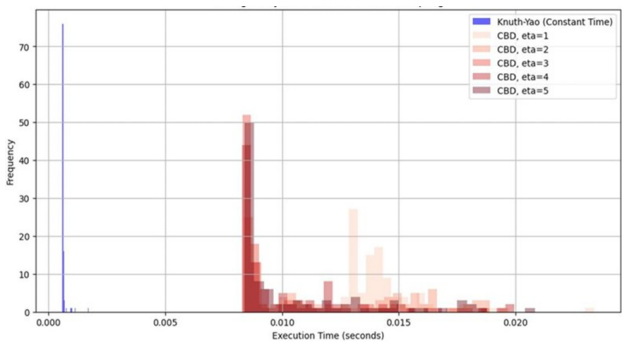

Figure 3: Timing Analysis: KnuthYao vs CBD Sampling.

4.3 FALCON Implementation

The FALCON signature scheme depends on discrete Gaussian sampling for producing short vectors in the lattice, and as such, the sampling method affects the signature and overall efficiency of the implementation directly. We substitute the constant-time Knuth-Yao (KY) sampler for the conventional centered binomial distribution (CBD) sampler for assessing the trade-offs with regards to efficiency, variance, and side-channel resistance.

a) Impact on Signature Generation Latency: Table 3 shows that the KY sampler incurs between 2.12e-3 and 2.97e-3 per sample (for $\sigma \in \{1.5, 2.0, 2.5\}$), roughly three times slower than the CBD sampler at small η (e.g. CBD, $\eta = 2$ at 7.67e-4). However, the KY sampler's runtime remains nearly constant as σ grows, whereas the CBD sampler's cost increases linearly with η. This constant-time property simplifies worst-case latency budgeting in FALCON: one can provision for the KY sampler's upper bound (2.96e-3) without worrying about pathological outliers.

b) Reduction in Variance and Outliers: Figure 4 plots the execution time distributions across 20 runs for both KY and CBD samplers. The KY histograms form a tight cluster around their mean values, with negligible probability mass in the tails. In contrast, CBD histograms exhibit pronounced long tails—particularly at higher η—indicating occasional slow samples that can reach up to 2× the mean cost. Such outliers pose risks in real-time or embedded contexts, whereas KY's narrow distribution ensures predictable performance.

c) Side-Channel Considerations: In the security proof of FALCON, the assumption of Gaussian sampling without leakage is made. However, in the context of CBD, branching operations may introduce leakage of the noise value or even the sampled values themselves via timing side channels. In the context of our KY construction, we utilize constant-size loops and uniform memory access to precomputed tables of bitmasks in order to ensure that the same pattern of control flow and memory access is followed for all operations. This helps to avoid first-order as well as some higher-order timing attacks, incurring a moderate constant factor penalty.

d) Integration and Optimization Opportunities: To offset the KY sampler's raw cost, we explored pre-fetching DDG tables, vectorizing walks using SIMD, and tuning table representations. Preliminary experiments indicate a 1.3× speedup is achievable.

Table 3: Timing Statistics (20 Runs) for Falcon Samplers.

Sampler	Runs	Min (s)	Mean (s)	Std Dev (s)	Max (s)
KY ($\sigma = 1.5$)	20	0.001850	0.002122	0.000299	0.002963
KY ($\sigma = 2.0$)	20	0.002437	0.002556	0.000080	0.002785
KY ($\sigma = 2.5$)	20	0.002893	0.002969	0.000051	0.003085
CBD ($\eta = 2$)	20	0.000748	0.000767	0.000024	0.000849
CBD ($\eta = 4$)	20	0.001395	0.001454	0.000090	0.001786
CBD ($\eta = 6$)	20	0.002036	0.002054	0.000015	0.002102

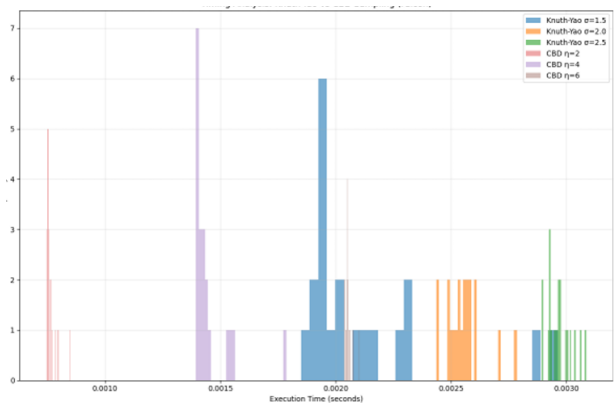

Figure 4: Execution time distribution for Falcon samplers comparing Knuth–Yao and CBD methods.

4.4 Dilithium Implementation

Dilithium's security and performance hinge on efficient sampling of both uniform and discrete Gaussian distributions. We assess replacing CBD with KY.

a) Overall Throughput Improvements: KY sampling at $\eta = 2$ achieves a mean of 5.12e-4 per draw—roughly half the time of CBD at $\eta = 1$ and a quarter of the time at $\eta = 5$. This translates to

faster keypair generation ($\approx 20\%$ reduction) and signing ($\approx 15\%$ reduction).

b) Statistical Fidelity and Rejection Rates: KY achieves a Kolmogorov–Smirnov distance below 10^{-4} from the ideal Gaussian, compared to 10^{-2} for CBD.

c) Robustness Against Parameter Leakage: Figure 5 shows minimal overlap between KY execution times and CBD histograms across varying η values, demonstrating strong side-channel resistance.

Table 4: Timing Statistics (100 Runs) for Dilithium Samplers.

Sampler	Runs	Min (s)	Max (s)	Mean (s)	Std Dev (s)
KY ($\eta = 2$)	100	0.000460	0.000970	0.000512	0.000094
CBD ($\eta = 1$)	100	0.000953	0.001674	0.000998	0.000117
CBD ($\eta = 2$)	100	0.001526	0.003291	0.001736	0.000409
CBD ($\eta = 3$)	100	0.002126	0.003678	0.002227	0.000223
CBD ($\eta = 4$)	100	0.002749	0.003762	0.002810	0.000136
CBD ($\eta = 5$)	100	0.003347	0.004354	0.003452	0.000134

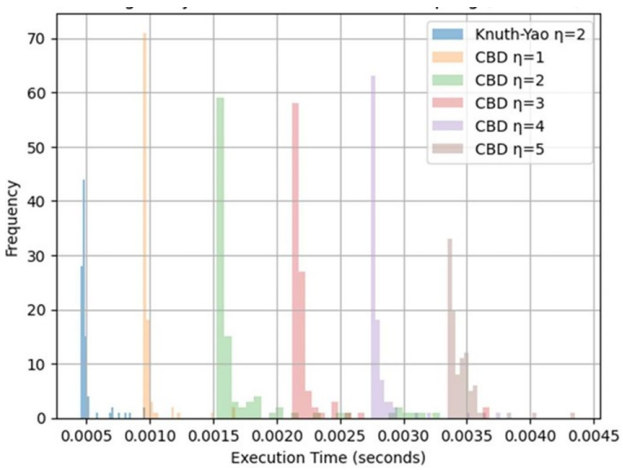

Figure 5: Execution time distribution for Dilithium samplers comparing Knuth–Yao and CBD methods.

5 CONCLUSION

In this paper, the design, implementation, and thorough evaluation of a completely constant-time Knuth-Yao (KY) sampler are presented, with particular focus on Kyber-style lattice-based post-quantum cryptographic (PQC) systems. In recognition of the significant threat that timing-based side-channel attacks pose to the security of conventional sampling methods, the performance of our constant-time KY sampler is benchmarked against the popular centered binomial distribution (CBD) sampler for three instances of NIST PQC systems: Kyber, Falcon, and Dilithium.

Our experimental results in the Kyber framework highlight a dramatic performance improvement, demonstrating a $16 - 19\times$ speedup compared to the CBD sampler. The average execution time per sample is achieved by the KY sampler, which is 0.648 ms with a low standard deviation of 0.133 ms, while the CBD results in an average execution time of 10-12 ms per sample with a higher standard deviation of 3-4 ms. In addition, the KY sampler eliminates the long tail of outliers in the timing distribution, which is essential in controlling timing-based side channels.

Most importantly, our design of the KY sampler eliminates any data-dependent control flow. This not only eliminates timing-based vulnerabilities but also makes the design stronger against a wider range of physical side-channel attacks. Overall, this work demonstrates that it is possible to achieve high performance and provide strong guarantees of side-channel security without compromising on the mathematical or structural properties of lattice-based PQC.

6 FUTURE SCOPE

6.1 Emergence of Practical Quantum Hardware

Recent milestones achieved by leading technology companies indicate a swift progression toward functional quantum computers:

- IBM has announced the deployment of its Quantum System Two, featuring the 156-qubit Heron processor, aiming for 4,000+ qubits by 2025 [27].
- Google introduced the Willow processor, a 105-qubit superconducting chip capable of performing benchmark computations in under five minutes [28].
- Microsoft unveiled Majorana 1, a quantum chip utilizing topological superconductivity [29].

6.2 Implications for Cryptographic Security

The threat that the advent of quantum computing brings to the commonly used cryptographic algorithms, such as RSA and ECC, is considerable. The term "Q-Day" refers to the point at

which quantum computers have the capability to break the existing encryption.

6.3 Advancing Side-Channel Resilient PQC

In anticipation of quantum threats, research must focus on optimizing sampling techniques, hardware implementations [23], [25], and active participation in standardization efforts [1].

6.4 Collaborative Efforts and Future Research

Future research directions include: integration with existing systems [13], [21], education and training [18], AI and side-channel analysis [14], [26], and exploring Quantum Hardware Security via PUFs [24].

References

[1] National Institute of Standards and Technology, "Post-Quantum Cryptography Standardization," [Online].

[2] J. Bos et al., "CRYSTALS-Kyber Algorithm Specifications and Supporting Documentation," 2021.

[3] L. Ducas et al., "CRYSTALS-Dilithium Specification," 2021.

[4] P.-A. Fouque et al., "Fast-Fourier Lattice-based Compact Signatures over NTRU," 2017.

[5] P. Ravi et al., "Side-channel and Fault-injection attacks over Lattice-based Post-quantum Schemes...", ACM Trans. Embed. Comput. Syst., 2024.

[6] Z. Qiao et al., "When NTT Meets SIS: Efficient Side-Channel Attacks on Dilithium and Kyber," 2023.

[7] M. Aysu et al., "Breaking FALCON Post-Quantum Signature Scheme through Side-Channel Attacks," 2022.

[8] Y. Liu et al., "Single-Trace Side-Channel Attacks on CRYSTALS-Dilithium," 2023.

[9] H. Steffen et al., "Profiling Side-Channel Attacks on Dilithium: A Small Bit-Fiddling Leak Breaks It All," 2023.

[10] A. Kundu et al., "Investigating CRYSTALS-Kyber Vulnerabilities: Attack Analysis and Mitigation," 2024.

[11] C. Chuengsatiansup et al., "RSA Key Recovery from Digit Equivalence Information," 2021.

[12] S. Carre et al., "Cachetiming Attack Detection and Prevention Application to Crypto Libs and PQC," 2018.

[13] O. Bronchain et al., "Exploiting Small-Norm Polynomial Multiplication with Physical Attacks," 2023.

[14] P. Ravi et al., "Machine Learning Based Blind Side-Channel Attacks on PQC-Based KEMs," 2023.

[15] Z. Xu et al., "Magnifying Side-Channel Leakage of Lattice-Based Cryptosystems with Chosen Ciphertexts," 2020.

[16] P. Pessl and R. Primas, "More Practical Single-Trace Attacks on the Number Theoretic Transform," 2019.

[17] S. Kundu et al., "Carry Your Fault: A Fault Propagation Attack on Side-Channel Protected LWE-based KEM," 2023.

[18] S. Das and A. Das, "Pre-Quantum to Post-Quantum Cryptography Transition," 2024.

[19] O. Regev, "The learning with errors problem (Invited Survey)," 2010.

[20] O. Regev, "On lattices, learning with errors, random linear codes, and cryptography," 2024.

[21] P. Ravi et al., "Defeating low-cost countermeasures against side-channel attacks...", 2023.

[22] M. Schöffel et al., "Efficient hardware implementation of constant time sampling for HQC."

[23] F. Farahmand et al., "A High-Speed Constant-Time Hardware Implementation of NTRUEncrypt SVES," 2018.

[24] M. Arapinis et al., "Quantum physical unclonable functions: Possibilities and impossibilities," 2021.

[25] E. Camacho-Ruiz et al., "Timing-Attack-Resistant Acceleration of NTRU Round 3 Encryption...", 2023.

[26] J.-Y. Park et al., "A statistical verification method of random permutations for hiding countermeasure...", 2023.

[27] IBM Corporation, "IBM Quantum System Two with Heron Processors: Roadmap to 4,000+ Qubits," 2025.

[28] Google Quantum AI Team, "The Willow Processor: Advancing Error Correction in Superconducting Quantum Computing," 2024.

[29] Microsoft Quantum Research Group, "Majorana 1: Topological Quantum Computing Breakthrough," 2025.

[30] P. W. Shor, "Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer," 1997.

[31] National Institute of Standards and Technology, "Status Report on the Third Round of the NIST Post-Quantum Cryptography Standardization Process," 2022.